



My Traceroute — Khám bệnh mạng toàn diện

Network Thực Chiến · Series: Debug Mạng từ A–Z · Tập 02

Nội dung

1. **MTR là gì?** — Sinh ra để giải quyết giới hạn của ping + traceroute
2. **So sánh MTR vs traceroute** — Tại sao MTR thắng?
3. **Cài đặt**
4. **Giao diện tương tác** — Đọc output realtime
5. **Cách đọc các chỉ số** — Loss%, Snt, Last, Avg, Best, Wrst, StDev
6. **Deep Dive: StDev** — Chỉ số quan trọng nhất
7. **Cheatsheet** — Report mode, TCP mode, các cờ thực chiến
8. **Kịch bản thực chiến** — Cách dùng MTR để chẩn đoán

Phần 1

MTR là gì?

Vấn đề với ping và traceroute

Khi mạng chậm hoặc chập chờn, kỹ sư thường dùng 2 lệnh kinh điển:

```
ping google.com          # Kiểm tra kết nối điề-m-điề-m
traceroute google.com    # Xem đường đi của gói tin
```

Nhưng cả hai đều có giới hạn:

	ping	traceroute
Thấy từng hop?	✗ Chỉ điểm đầu–cuối	✓ Có
Phát hiện chập chờn?	⚠ Cần chờ lâu	✗ Quét 1 lần, bỏ sót
Realtime?	✗ Phải đọc từng dòng	✗ Dừng lại sau khi xong
Thống kê Loss%?	⚠ Phải tự đếm	✗ Không có

Cần một công cụ **kết hợp cả hai**, chạy liên tục, và có thống kê chính xác.

MTR = ping + traceroute + realtime dashboard

MTR (My Traceroute) ra đời để giải quyết đúng vấn đề đó.

traceroute: Chụp một bức ảnh tĩnh về đường truyền

```
Hop 1: 192.168.1.1    1ms
Hop 2: 103.7.x.x      8ms
Hop 3: 1.2.3.4        22ms
[XONG – không cập nhật nữa]
```

mtr: Quay một đoạn video liên tục về đường truyền

Host	Loss%	Snt	Last	Avg	Best	Wrst	StDev	
192.168.1.1	0.0%	120	0.5	0.6	0.4	1.2	0.1	
103.7.x.x	0.0%	120	7.8	8.1	7.2	15.3	0.8	
1.2.3.4	2.3%	120	22.1	23.4	21.5	89.7	12.4	← bottleneck!

[CẬP NHẬT MỖI GIÂY – thấy được chập chờn ngẫu nhiên]

MTR vs traceroute — So sánh chi tiết

Tiêu chí	<code>traceroute</code>	<code>mtr</code>
Cách hoạt động	TTL tăng dần, quét 1 lần, dừng	Probe liên tục mỗi giây, cập nhật realtime
Phát hiện chập chờn	✗ Rất kém — bỏ sót lỗi ngẫu nhiên	✓ Tuyệt vời — Loss% chính xác theo thời gian
Giao diện	Static text, in từng dòng	Dashboard realtime hoặc Report mode
Thống kê	Không có	Loss%, Avg, Best, Wrst, StDev
Ứng dụng chính	Xác định nhanh routing path	Chẩn đoán độ ổn định của kết nối

💡 Tóm lại:

`traceroute` = **ảnh chụp** đường truyền

`mtr` = **video quay liên tục** đường truyền

Phần 2

Cài đặt & Bắt đầu

Cài đặt

```
# Ubuntu / Debian  
sudo apt update && sudo apt install mtr
```

```
# CentOS / RHEL  
sudo yum install mtr
```

```
# macOS (Homebrew)  
brew install mtr  
# ⚠️ macOS yêu cầu quyền root:  
sudo mtr google.com
```


Giao diện tương tác — Lệnh cơ bản

```
mtr google.com
```

Output realtime (cập nhật mỗi giây):

```
My traceroute [v0.95]
Keys: H=Help D=Display mode R=restart S=quit

Host      Loss%  Snt   Last    Avg    Best  Wrst  StDev
1. 192.168.1.1    0.0%  120    0.5     0.6    0.4    1.2    0.1
2. 103.7.96.1     0.0%  120    7.8     8.1    7.2    9.3    0.4
3. 27.68.226.1    0.0%  120   10.2    10.5    9.8   12.1    0.5
4. ???          ---     0     ---     ---     ---     ---     ---
5. 142.251.49.14  0.0%  120   11.1    11.3   10.8   14.2    0.6
6. 142.250.196.46 0.0%  120   11.8    12.1   11.2   15.3    0.7
```

Nhấn **q** để thoát.

Phần 3

Cách đọc các chỉ số MTR

Ý nghĩa các cột

Cột	Ý nghĩa
Host	IP hoặc hostname của hop (router trung chuyển)
Loss%	% gói tin bị mất tại hop này
Snt	Số gói đã gửi (Sent) — càng nhiều, thống kê càng chính xác
Last	RTT của gói tin gần nhất (ms)
Avg	RTT trung bình của tất cả gói đã gửi (ms)
Best	RTT thấp nhất từng ghi được (ms)
Wrst	RTT cao nhất từng ghi được (ms)
StDev	Độ lệch chuẩn — đo mức chậm chờn của kết nối
???	Hop không phản hồi ICMP — không phải lỗi nếu các hop sau bình thường

Quy tắc vàng đọc Loss%

⚠ Đây là lỗi hay không phải lỗi?

Host	Loss%	Snt	Last	Avg	Best	Wrst	StDev	
1. 192.168.1.1	0.0%	100	0.5	0.6	0.4	1.2	0.1	
2. 103.7.96.1	0.0%	100	7.8	8.1	7.2	9.3	0.4	
3. 27.68.226.1	100.0%	100	---	---	---	---	---	← ⚠ Đáng ngờ?
4. 142.251.49.14	0.0%	100	11.1	11.3	10.8	14.2	0.6	← ✓ Bình thường!
5. 142.250.196.46	0.0%	100	11.8	12.1	11.2	15.3	0.7	

Hop 3 có Loss 100% nhưng KHÔNG PHẢI LỖI.

Router tại Hop 3 đang **Rate-Limit** gói ICMP gửi đến CPU của nó — hành vi bình thường để tự bảo vệ.

Chỉ lo khi **hop cuối cùng (đích)** có Loss% > 0.

Quy tắc: Tìm hop **đầu tiên** có Loss% cao **VÀ** các hop sau cũng có Loss% cao → đó mới là bottleneck thực sự.

Ví dụ thực chiến — Đọc output

Host	Loss%	Snt	Last	Avg	Best	Wrst	StDev	
1. 192.168.1.1	0.0%	200	0.5	0.6	0.4	1.2	0.1	← Router nhà: OK ✓
2. 103.7.96.1	0.0%	200	7.8	8.1	7.2	9.3	0.4	← ISP hop 1: OK ✓
3. 103.7.100.5	3.5%	200	15.2	18.7	14.1	89.3	22.1	← ! Loss + StDev cao
4. 27.68.226.1	3.5%	200	15.8	19.1	14.5	91.2	23.4	← Loss giữ nguyên
5. 142.250.196.46	3.5%	200	16.1	19.4	14.8	92.0	24.1	← Đích: Loss = 3.5%

Chẩn đoán:

- Loss xuất hiện từ **Hop 3** và giữ nguyên đến đích → vấn đề nằm tại **Hop 3**
- StDev = 22ms ở Hop 3 → rất chậm chờn
- Wrst = 89ms trong khi Best = 14ms → ping nhảy lên 6 lần

→ **Kết luận:** Link giữa Hop 2 và Hop 3 bị nghẽn hoặc lỗi phần cứng.



Phần 4

Deep Dive: Chỉ số StDev

StDev là gì?

StDev (Standard Deviation) = Độ lệch chuẩn, dùng để **xấp xỉ Jitter**.

⚠ Lưu ý: Jitter chuẩn (PDV — Packet Delay Variation) đo biến thiên giữa các gói liên tiếp.

StDev đo phân tán so với trung bình (Avg).

Hai khái niệm liên quan nhưng không giống nhau — MTR dùng StDev vì dễ tính online.

Công thức:

$$\sigma = \sqrt{[\Sigma(x - \mu)^2 / N]}$$

x = RTT từng gói tin

μ = Avg (RTT trung bình)

N = Số gói đã gửi

Ý nghĩa đơn giản: StDev càng nhỏ → ping càng đều → mạng càng ổn định.

StDev — Ví dụ minh họa

Trường hợp 1: Mạng ổn định ✓

Ping lần lượt: 20ms, 21ms, 19ms, 20ms, 21ms

Avg = 20.2ms

StDev $\approx$ 0.8ms ← Rất thấp → mạng mượt

Trường hợp 2: Mạng chập chờn ✗

Ping lần lượt: 20ms, 150ms, 18ms, 200ms

Avg = 97ms

StDev $\approx$ 81ms ← Rất cao → mạng trời sập thất thường

Kết luận thực chiến:

Đường truyền **Ping cao nhưng StDev thấp** (ổn định) **tốt hơn**
đường truyền **Ping thấp nhưng StDev cao** (chập chờn).

StDev cao = tín hiệu xấu cho VoIP, Game online, video call.

StDev cao — Nguyên nhân thường gặp

Host	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. 192.168.1.1	0.0%	200	0.5	0.6	0.4	1.2	0.1
2. 103.7.96.1	0.0%	200	8.1	8.3	7.2	9.3	0.4
3. core-router.isp	0.0%	200	11.2	45.3	9.8	312.0	87.6
4. 142.250.196.46	0.0%	200	12.1	46.1	10.2	315.3	88.1

← StDev = 87ms!

StDev vọt lên tại một hop cụ thể thường do:

Nguyên nhân	Dấu hiệu đi kèm
Router CPU overload	Loss% thấp nhưng Wrst rất cao
Queueing delay (buffer bloat)	Avg cao hơn Best rất nhiều
Link bị lỗi / interference	Loss% tăng cùng với StDev
Traffic policing / shaping	StDev cao ở giờ cao điểm

Phần 5

Cheatsheet Thực chiến

Các lệnh quan trọng

1. Giao diện tương tác cơ bản

```
mtr google.com
```

2. Report mode – gửi cho NOC/ISP

```
mtr -r google.com          # 10 vòng (mặc định) rồi dừng
```

```
mtr -rw google.com         # -w: không cắt ngắn hostname dài
```

```
mtr -r -c 100 google.com   # 100 vòng để thống kê chính xác hơn
```

3. Không resolve DNS – tránh hiển thị chậm

```
mtr -n google.com
```

4. TCP mode – bypass firewall chặn ICMP

```
sudo mtr -T -P 443 google.com # Test qua port HTTPS
```

```
sudo mtr -T -P 80 google.com  # Test qua port HTTP
```

5. Tăng tần suất probe – phát hiện lỗi nhanh hơn

```
sudo mtr -i 0.1 google.com    # 10 gói/giây (mặc định 1/giây)
```

Report Mode — Output mẫu

```
mtr -rw google.com
```

```
Start: 2026-04-24T10:00:00+0700
```

```
HOST: myserver
```

		Loss%	Snt	Last	Avg	Best	Wrst	StDev
1.	-- 192.168.1.1	0.0%	10	0.5	0.6	0.4	1.2	0.1
2.	-- 103.7.96.1	0.0%	10	7.8	8.1	7.2	9.3	0.4
3.	-- ???	---	0	---	---	---	---	---
4.	-- 142.251.49.14	0.0%	10	11.1	11.3	10.8	14.2	0.6
5.	-- 142.250.196.46	0.0%	10	11.8	12.1	11.2	15.3	0.7

💡 Dùng `-rw` thay vì `-r` để tránh bị cắt ngắn hostname.

Dùng `-c 100` để tăng số vòng → thống kê chính xác hơn khi gửi cho ISP.

TCP Mode — Khi ICMP bị chặn

Firewall doanh nghiệp hoặc cloud thường **chặn ICMP** nhưng mở TCP:

ICMP mode (mặc định):

```
mtr google.com
```

```
→ Hop 3: ??? ← Firewall chặn ICMP
```

```
→ Hop 4: ???
```

```
→ Hop 5: 142.250.x.x OK
```

TCP mode (bypass firewall):

```
sudo mtr -T -P 443 google.com
```

```
→ Hop 3: 27.68.x.x 8ms ← Thấy được hop ẩn!
```

```
→ Hop 4: 142.251.x.x 10ms
```

```
→ Hop 5: 142.250.x.x 11ms
```

Khi nào dùng TCP mode?

- Nhiều hop hiển thị **???** liên tiếp
- Đích reach được (site tải được) nhưng mtr không thấy path
- Debug đường đi thực tế của HTTP/HTTPS traffic

Phần 6

Kịch bản thực chiến

Scenario A: "Mạng chậm, không biết lỗi ở đâu"

```
mtr -r -c 100 -w google.com
```

Đọc output theo thứ tự:

1. Tìm hop đầu tiên có Loss% > 0
→ Đó là nguồn gốc vấn đề
2. Kiểm tra StDev tại hop đó
→ StDev cao + Loss cao = link bị lỗi
→ StDev cao + Loss = 0 = CPU/queue overload
3. So sánh Avg với Best
→ Avg >> Best tại một hop = queueing delay (buffer bloat)
4. Nếu tất cả hop = 0% Loss nhưng vẫn chậm
→ Vấn đề ở application layer (không phải mạng)
→ Dùng curl -w để đo TTFB

Scenario B: "Video call bị giật, ping trung bình ổn"

```
# Chạy lâu để bắt được spike ngẫu nhiên  
sudo mtr -i 0.2 -c 500 -rw meet.google.com
```

Tìm kiếm:

- `StDev > 20ms` tại bất kỳ hop nào → Jitter cao → gây giật video/audio
- `Wrst >> Best` (ví dụ Best=10ms, Wrst=500ms) → spike ngẫu nhiên

Xử lý:

Vấn đề	Giải pháp
StDev cao tại hop đầu (router nhà)	Kiểm tra WiFi interference, đổi sang dây LAN
StDev cao tại hop ISP	Báo ISP kèm output <code>mtr -rw</code>
StDev cao nhiều hop cuối	Vấn đề phía server/CDN, thử region khác

Scenario C: Gửi report cho NOC / ISP

```
# Chạy từ cả 2 chiều (bidirectional test)

# Từ máy của bạn đến đích:
mtr -rw -c 100 target.server.com > report_outbound.txt

# Nhờ server đích chạy ngược lại về máy bạn:
mtr -rw -c 100 your.public.ip > report_inbound.txt
```

⚠ Quan trọng: Mạng không đối xứng (asymmetric routing).

Đường đi **ra** và đường đi **về** có thể đi qua các router hoàn toàn khác nhau.

Vấn đề chỉ xuất hiện 1 chiều → phải test cả 2 chiều mới tìm đúng.

Khi liên hệ ISP, luôn đính kèm:

- Output `mtr -rw` (2 chiều)
- Thời điểm xảy ra sự cố
- IP nguồn và đích

Key Takeaways

Tình huống	Dấu hiệu	Kết luận
Một hop Loss 100%, hop sau = 0%	ICMP rate-limit	Không phải lỗi
Loss xuất hiện tại hop X, giữ đến đích	Gói tin bị drop	Lỗi tại hop X
StDev cao tại một hop	Ping trời sục	CPU overload / queue
Avg >> Best tại một hop	Latency cao ngẫu nhiên	Buffer bloat
Tất cả hop OK, vẫn chậm	Mạng ổn	Vấn đề ở application

Bộ lệnh cốt lõi:

```
mtr google.com           # Realtime dashboard
mtr -rw -c 100 host       # Report gửi NOC/ISP
sudo mtr -T -P 443 host    # Bypass firewall ICMP
sudo mtr -i 0.1 host       # Phát hiện spike nhanh
```

Cảm ơn đã theo dõi! 🙏

Network Thực Chiến

Tập tiếp theo: **ss** — X-quang hệ thống socket

"Ping cao nhưng StDev thấp tốt hơn Ping thấp nhưng StDev cao."